

Retrieval Augmented Generation

Generative AI

Module 3 – Lesson 3

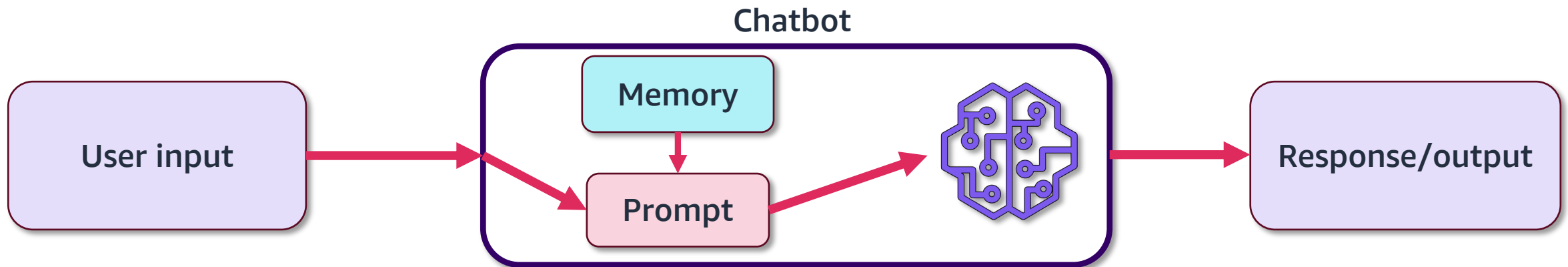
Today's activities



- Review of lesson 2
- Grounding a model
- Retrieval Augmented Generation (RAG)
- RAG Workflow
- Multimodal Embeddings
- Multimodal RAG

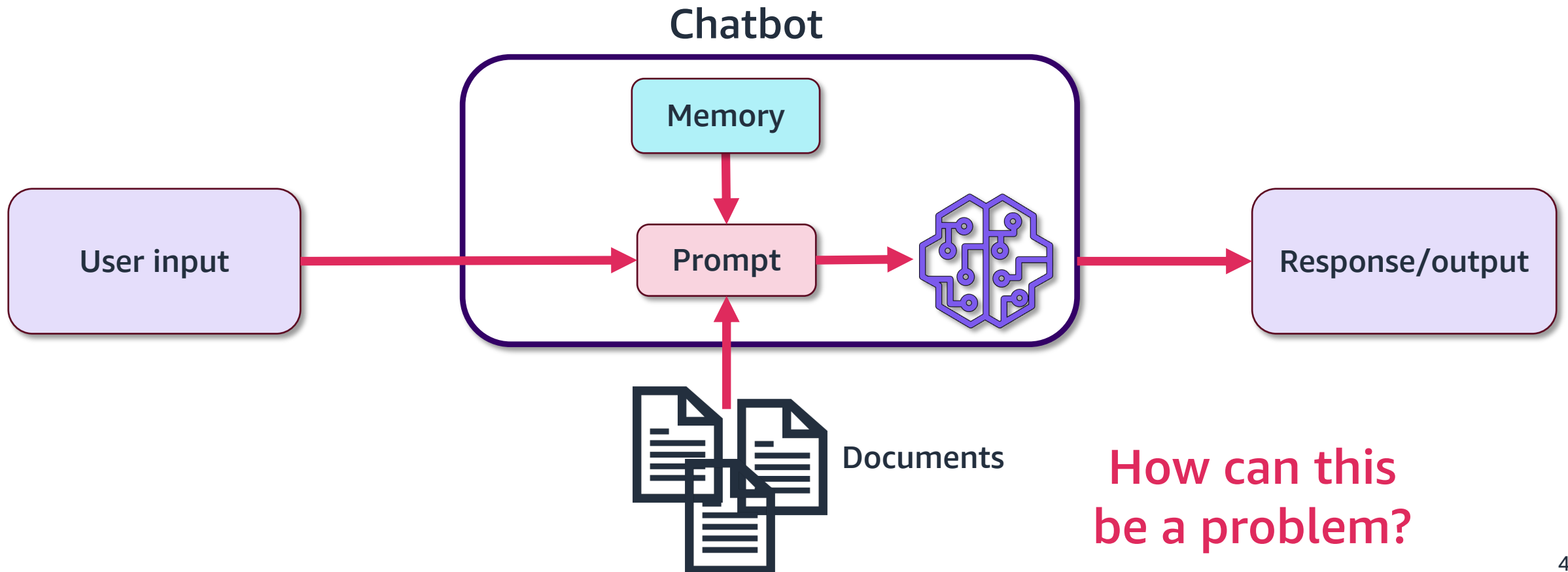
Review: Chatbot components

- The simplest chatbot only requires:
 - **A large language model (LLM)**
 - **Prompt template**
 - Define the role of the chatbot and general guidelines to follow
 - **Memory module**
 - Persist information across conversations



Review: Chatting with Documents

- Prompt with the entire text of the document(s)



Grounding a model

'Grounding' a model

Grounding encompasses a series of techniques to make the LLM's output consistent with:

- ⚙ Established facts (mitigate hallucinations)
- ⚙ Common-sense reasoning
- ⚙ Real-world context
- ⚙ User intent

Grounding

Grounding can be achieved via:

- ⚙ Prompt engineering (in-context learning)
- ⚙ Fine tuning
- ⚙ RAG (Retrieval Augmented Generation)
- ⚙ Tools (with agents)
- ⚙

Overview of fine-tuning LLMs

- Fine-tuning is a continuation of **pre-training**
- Fine-tuning requires supplementing pre-training with very **high-quality training data**
- Fine-tuning an LLM is **customizing** it to perform specialized tasks or to adapt to specific domains
- Fine-tuning changes the model weights and it creates a **new model**

Strengths of fine-tuning

- **Easy to use** Amazon Bedrock UX
 - No coding required
 - Upload CSV files to S3 and press buttons
 - Storage and inference managed by Amazon Bedrock
- It only requires **few high-quality training examples** (~100 up to ~1,000). Fine-tuning dataset does not need to be huge, but it needs to be of very **high quality**.
- Good match for data that changes perhaps ~annually



Weaknesses of fine-tuning (1/2)

- **Poor fit for fast-changing** information on the timescale of ~days, weeks
 - Example: stock prices
- Poor fit for improving performance on weaknesses like math
 - One **cannot fix fundamental limitations** of LLMs with fine-tuning



Weaknesses of fine-tuning (2/2)

- Imposes **maintenance** burden on teams
 - Update fine-tuning training data when outdated or irrelevant
 - Parent model changes may force you to re-fine-tune on short notice
- May **impact Responsible AI** mitigations
 - Fine-tuned models might perform worse in terms of safety, privacy than baseline models
 - You need to re-evaluate performance after fine-tuning and check for model drift



Retrieval Augmented Generation (RAG)

Retrieval augmented generation: Overview

- **In-context learning** technique
 - Provide knowledge to LLMs beyond that acquired during training
 - No training or fine-tuning

Retrieval augmented generation: Steps

- Three steps:
 1. Retrieve data from an external source
 2. Augment the context of the prompt with retrieved data
 3. Generate response with LLM, based on the prompt and retrieved data
- The data may come from different sources:
 - Internal documents
 - Logs
 - Company policies...

Fine-tuning vs RAG

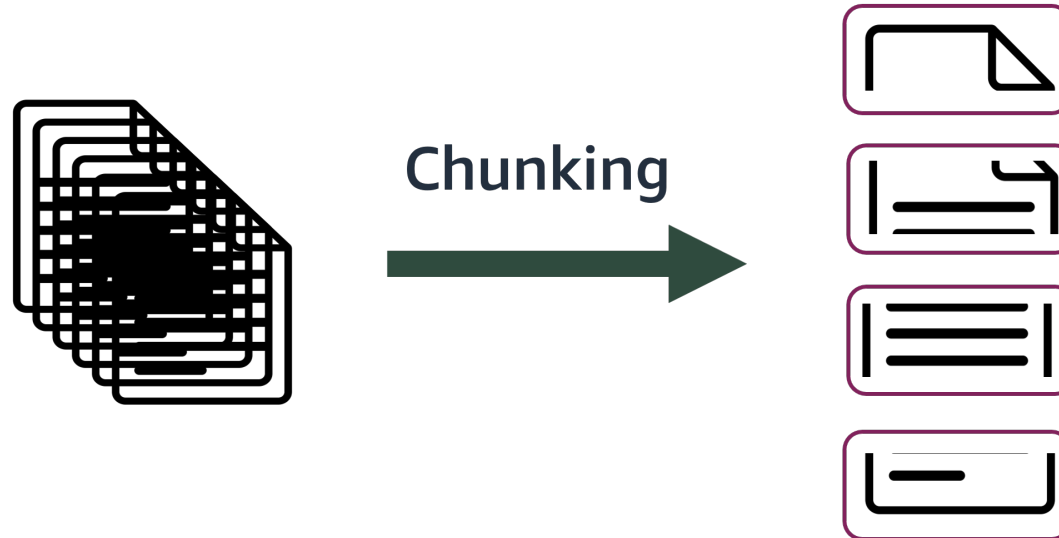
- While both techniques are suitable for **domain adaptation**, there are some key differences.

	Fine-tuning	RAG
Rationale	Create a task-specific LLM	Provide LLM with task-specific context
How to apply	Adapt the model weights for a task through gradient updates (training)	Retrieve relevant data from external sources while prompting
Unique value	Want to retain knowledge within the model	Want to retrieve current or fluctuating data
Example	Fine-tune an LLM on academic content to create a virtual tutor	Ask LLM ‘Is it a good time to buy stocks of a given company?’

RAG Workflow

Document chunking

- Typically, long documents are **split into chunks** to fit into an LLM's context window.
- Vectorized representations of chunks are stored in a **vector database**.
- Query is converted to vector → Find similar chunks from vector DB.

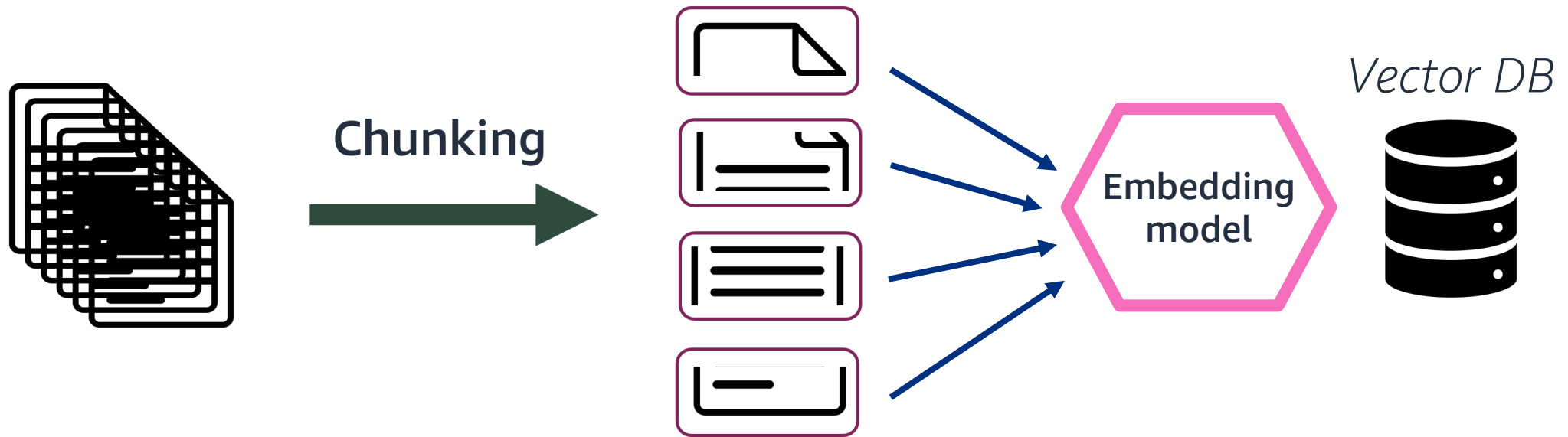


Tips while chunking

- ⚙️ **Overlap**: Allows for continuity between chunks.
- ⚙️ **Size of Chunks**: Maximum size of chunks
 - » Too small: Not enough context in each chunk
 - » Too large: May exceed context window of the LLM
- ⚙️ **Separators**: Enables chunking based on multiple separators
- ⚙️ **Some Text Splitters offered by LangChain**:
 - » **CharacterTextSplitter**: Split based on characters
 - » **MarkdownHeaderTextSplitter**: Split based on specific set of headers
 - » **RecursiveCharacterTextSplitter**: Split based on sequence of separators

Vector Database

- Retrieval requires fast fetching of relevant content
- Store vectorized representations of all chunks in a database
 - **Vector Database**
- Convert query to vector → Find similar chunks from vector database
 - Similarity search of embeddings



Increasing LLM recall

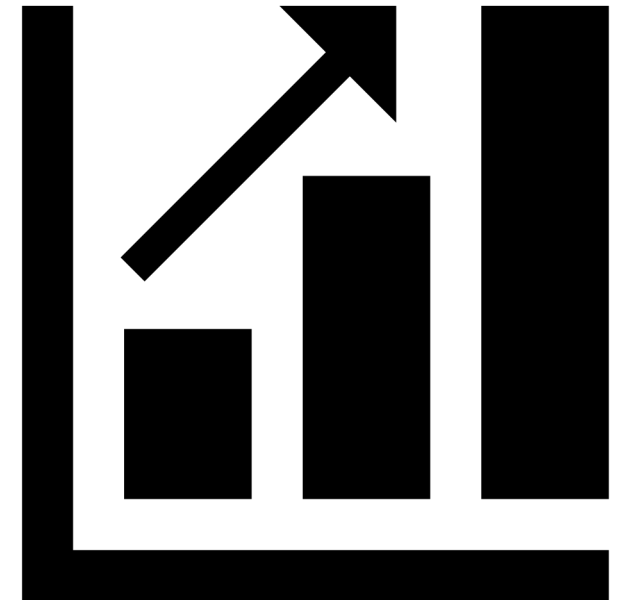
- Ability of the LLM to find appropriate information from all the context shared



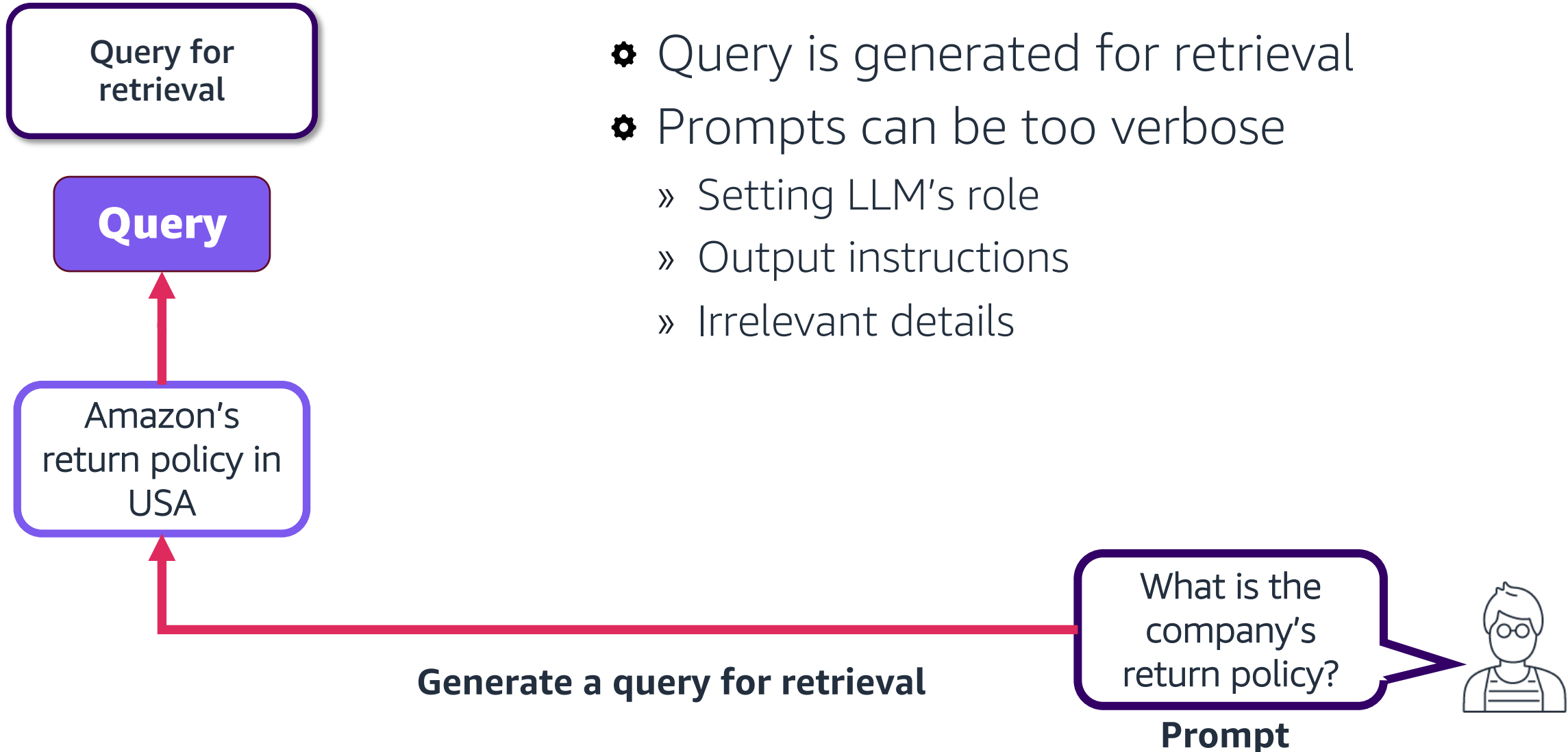
- **Solution:** Increase the number of retrieved documents
 - Are all documents equally important?

Rerankers

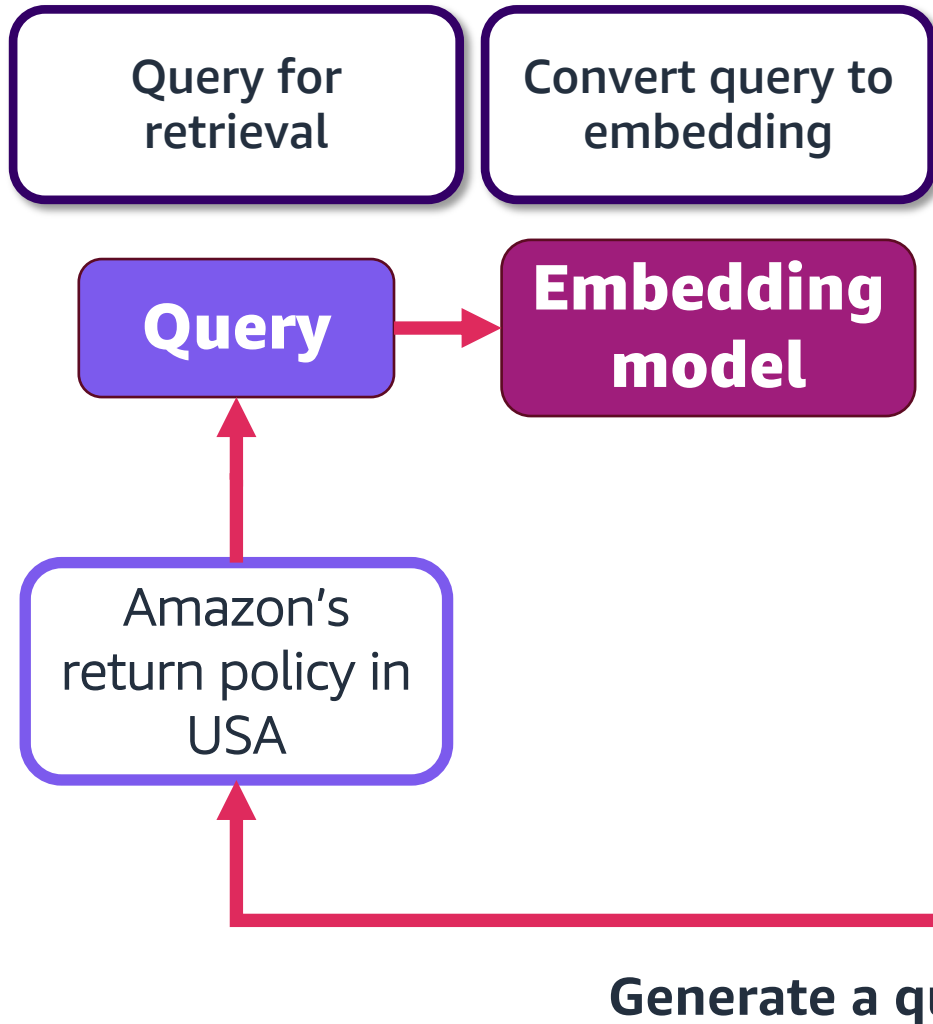
- Specialized models that compute **similarity score** between documents and query
 - Much more **accurate** than embedding models
 - But much **slower**
- Two-stage retrieval
 - Retrieve relevant documents using embeddings
 - Measure relevance using rerankers



RAG architecture (1/5)

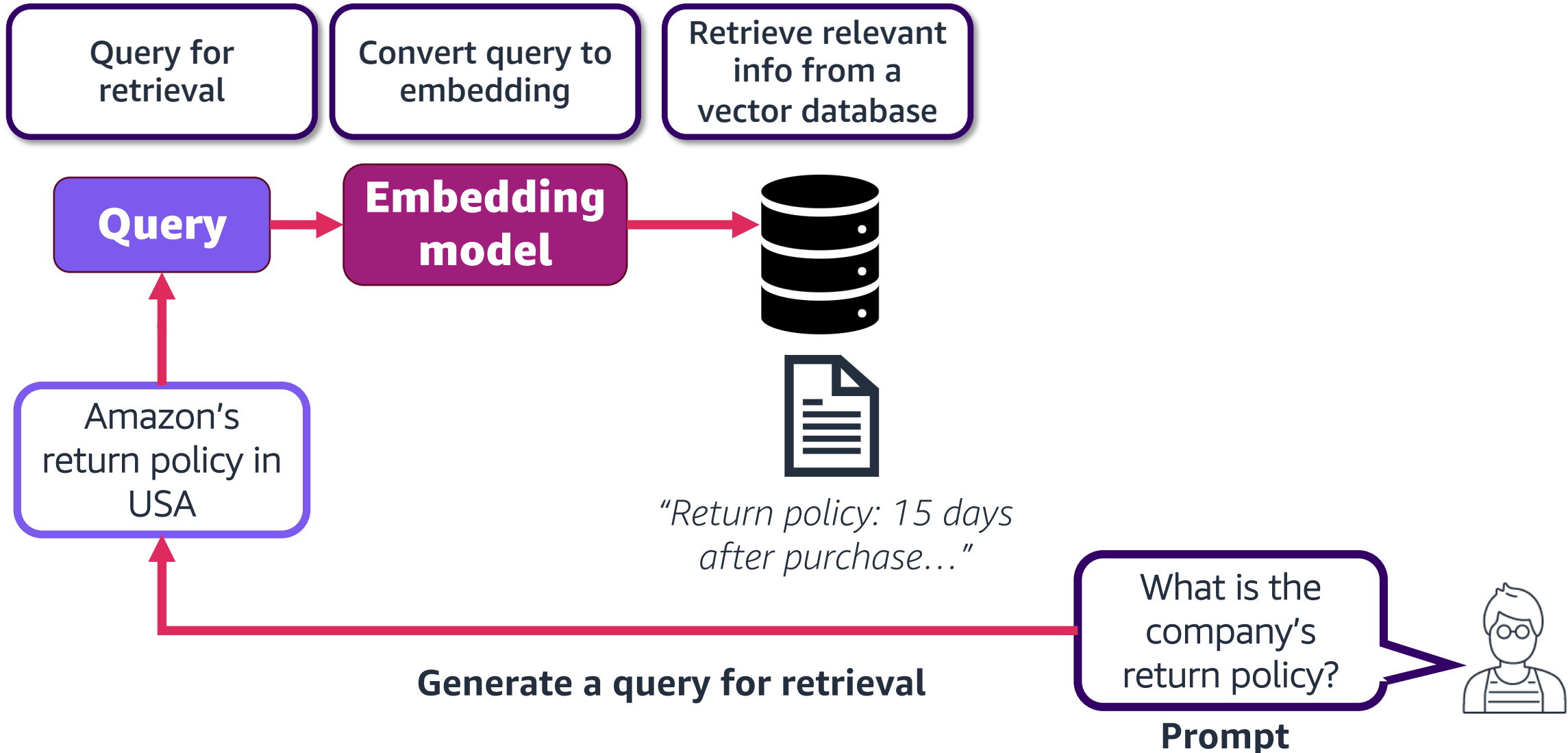


RAG architecture (2/5)

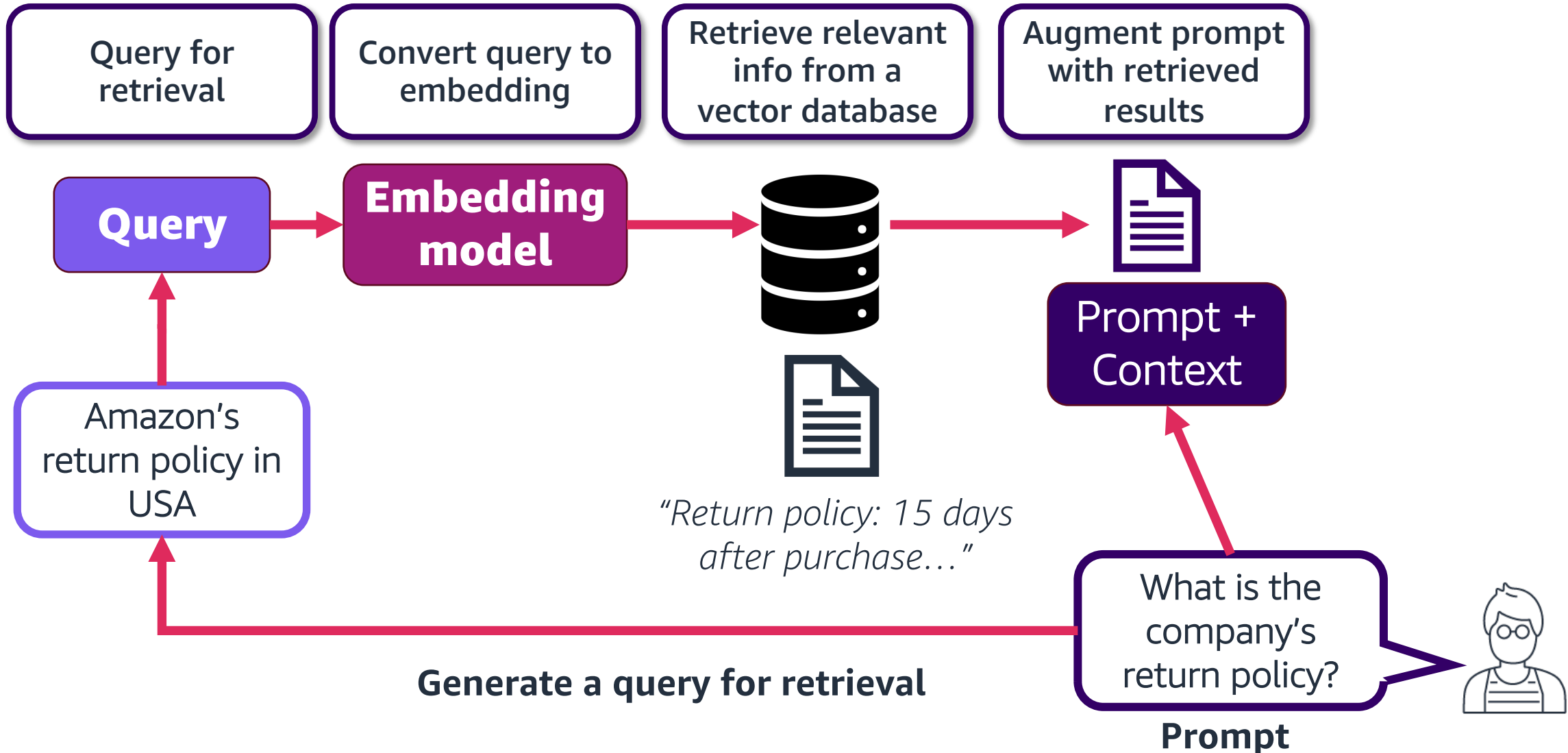


- ⚙ Common embedding model
 - » Vector database
 - » Query

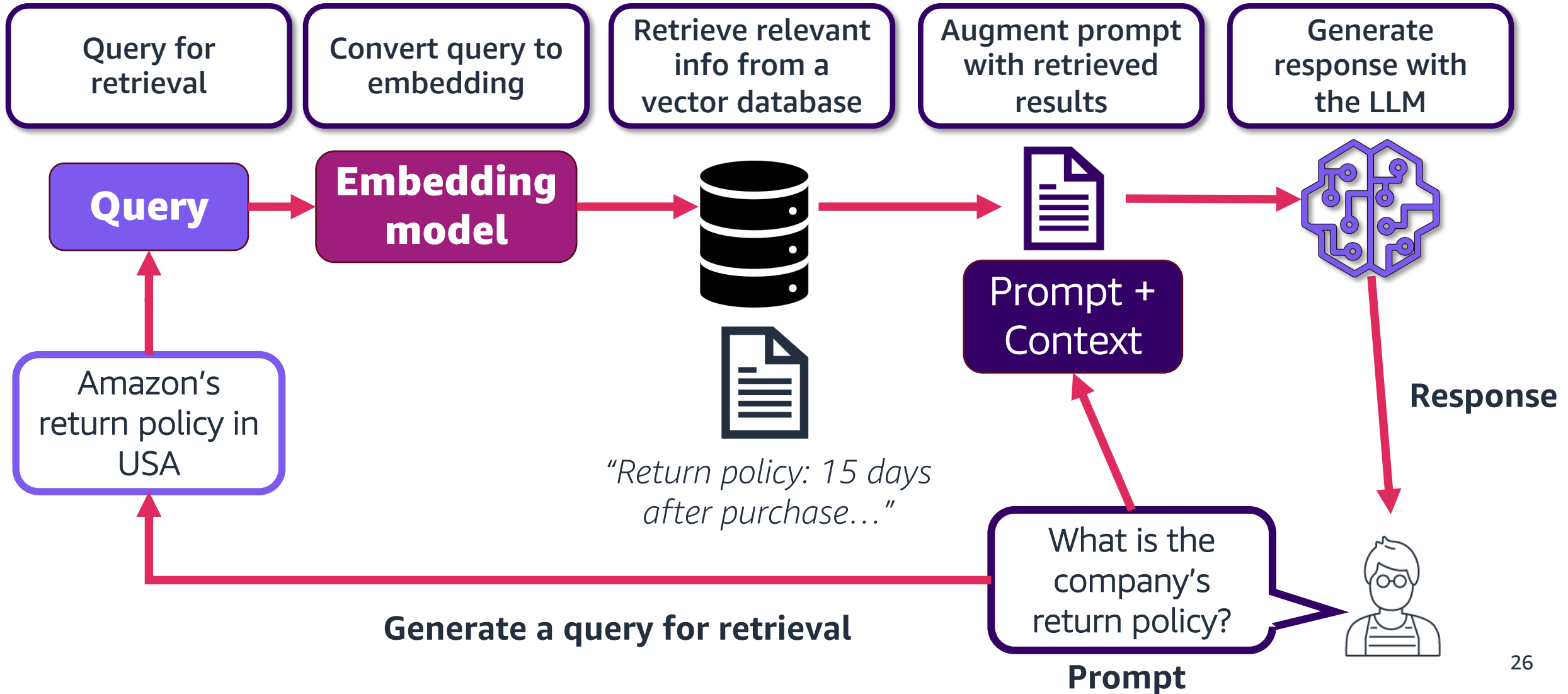
RAG architecture (3/5)



RAG architecture (4/5)



RAG architecture (5/5)



Multimodal Embeddings

Challenges with multimodal models: Representation

- Learning how to **represent multiple modalities efficiently**
 - Exploit complementary information from modalities
 - Avoid redundancy from modalities
- **Combine the data** from heterogeneous sources
 - Language is symbolic
 - Images are numeric matrices
 - Audio and video are represented as signals
- Some modalities will require you to **handle noise or missing data**

Challenges with multimodal models: Alignment

- Alignment is about **integrating information from different modalities**
 - Identifies direct relations between elements, across multiple modalities
 - Example: Aligns the elements of an image and its caption

Challenges with multimodal models: Translating

- **Mapping** one modality to another
- What if data is **heterogenous**?
 - Image-to-text
 - Text-to-image
- Relationship between them could be **subjective or open-ended**
 - There can be several accurate descriptions or interpretations of a painting

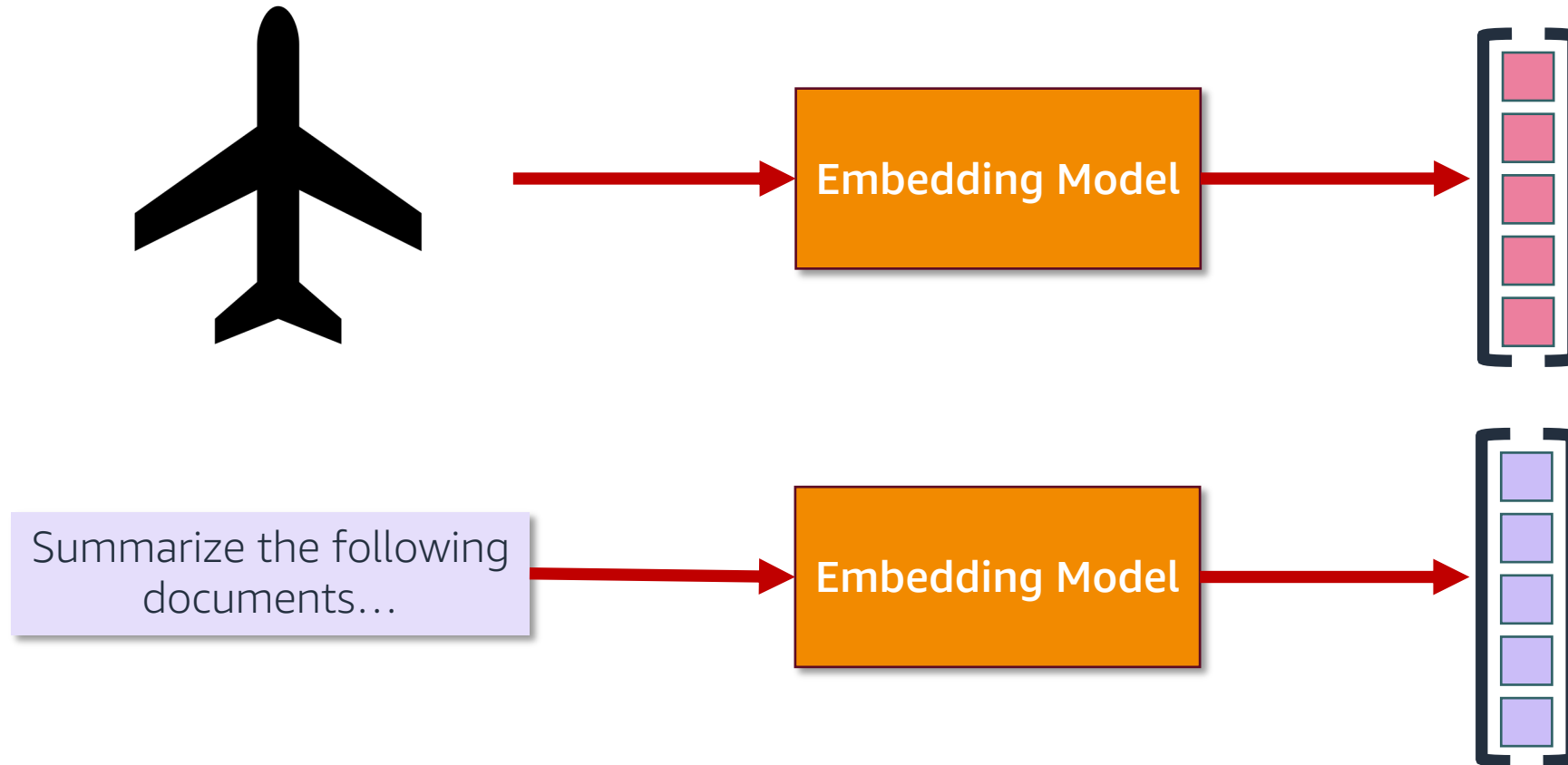
How to address this challenge?

- The model should know that an image, and its corresponding description (caption) is about the same underlying concept
 - Ex. “digital camera”
- How does the model encode this information while integrating both modalities?

Multimodal embeddings

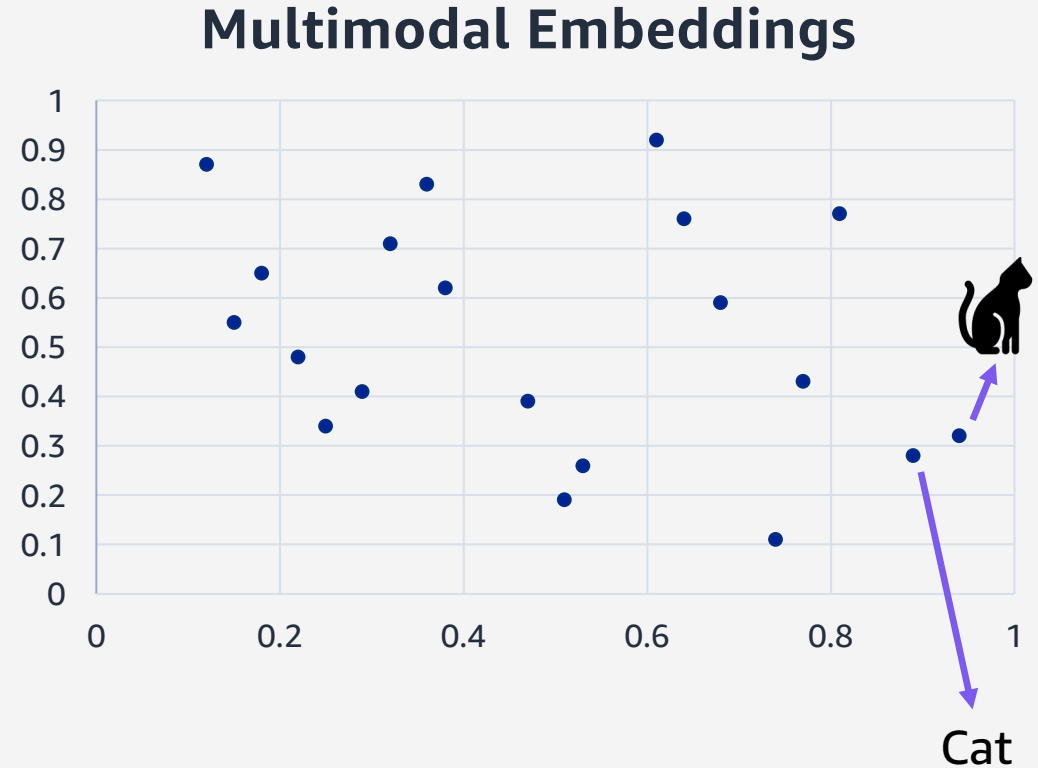
Multimodal embeddings: Representation

- Represent multiple modalities using same embedding model



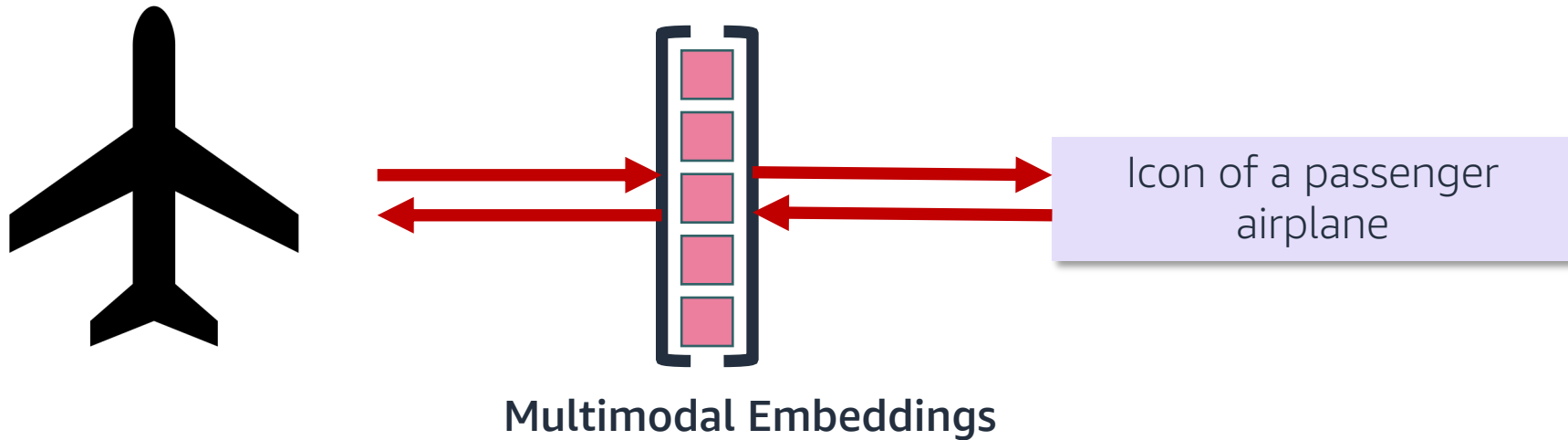
Multimodal embeddings: Alignment

- Joint embedding space of a multimodal model that understands both text and images
- Objects that are similar are closer together and dissimilar objects are farther apart
- The model preserves semantic similarity within and across modalities



Multimodal Embeddings: Translation

- Map content in one modality to another
 - Cross-modality translation



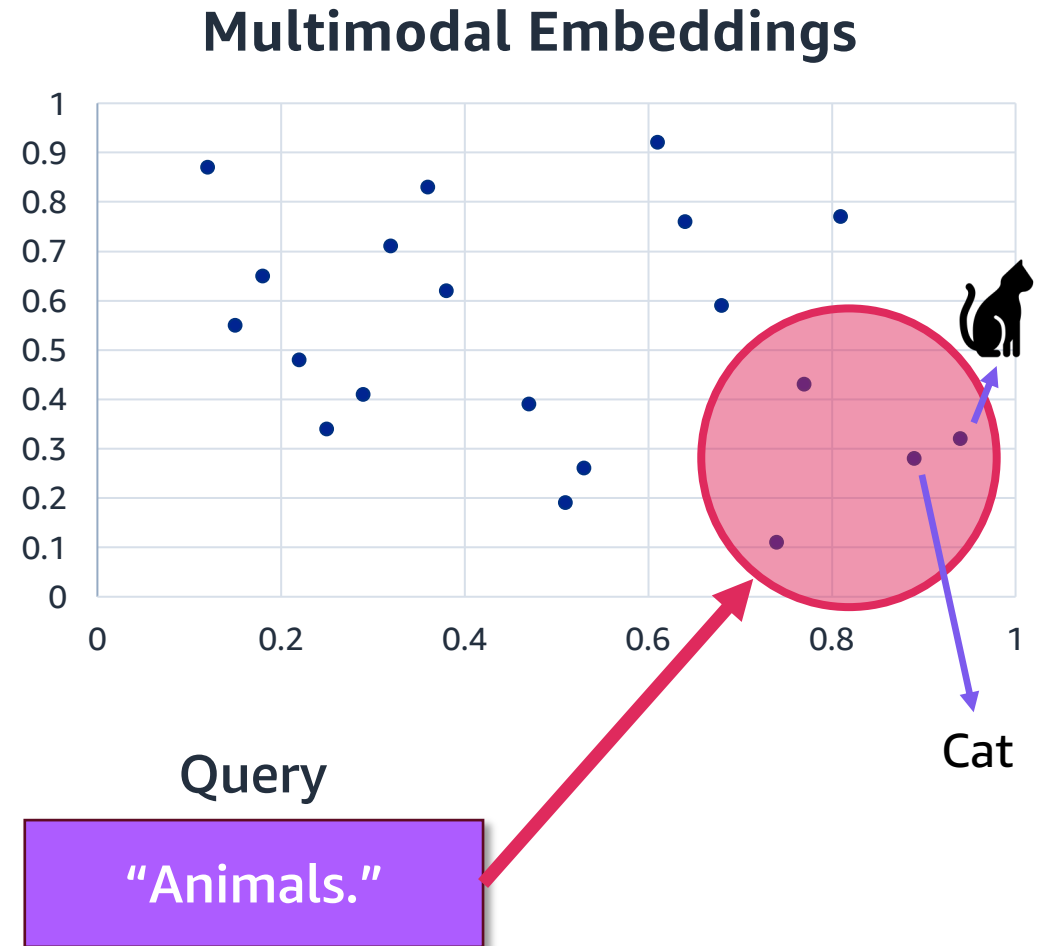
Multimodal RAG

How do we perform cross-modal retrieval?

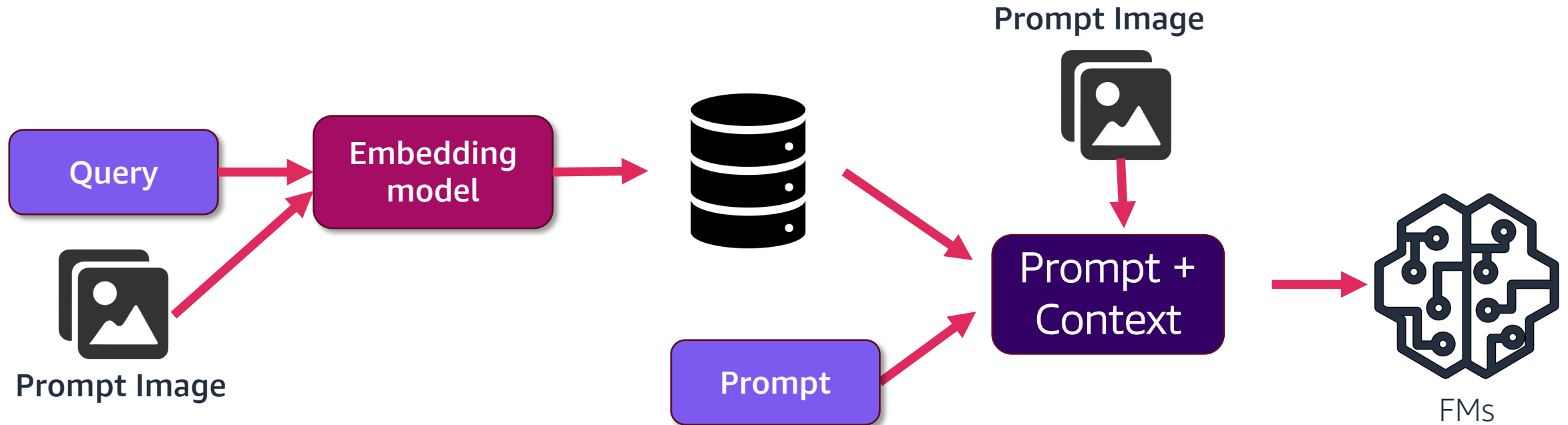
- ⚙ Create text descriptions of all other modalities and perform lexical search
 - » What if similar content doesn't contain similar words/phrases?
 - » **Solution:** Multimodal embeddings extend to all modalities

Multimodal search and retrieval

- Retrieve embeddings close to the query word/phrase.
- Search and retrieve across all modalities



Multimodal RAG workflow



- Use the same embedding model for the vector database and embedding the query
- Images can be used for retrieval, prompting and both

RAG Summary

Strengths of RAG

- Improved **factual accuracy** of generated text
 - Can help mitigate hallucinations (although it might not be able to *fully* avoid them)
- **Adaptability** of one same model to multiple domains
 - Replacing the knowledge sources allows to adapt the model to different use cases (internal documentation, customer policies, knowledge of specialized domains, etc.)
- Easier maintainability of the system for **evolving information**
 - Refreshing the knowledge source keeps information up to date. Best suited for facts that change e.g. quarterly, i.e. not efficient if facts change daily
- Increased **explainability and interpretability** of outputs
 - The generated responses can be traced back to the actual information sources



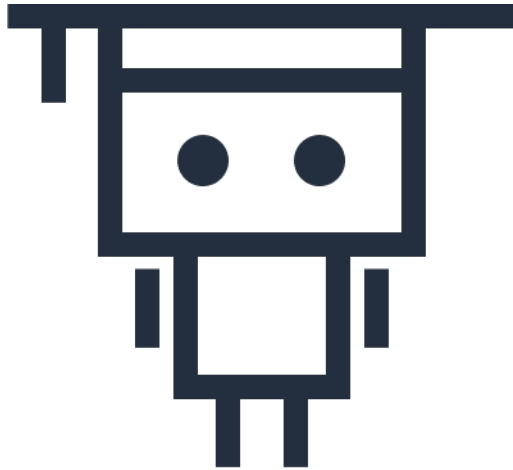
Weaknesses of RAG

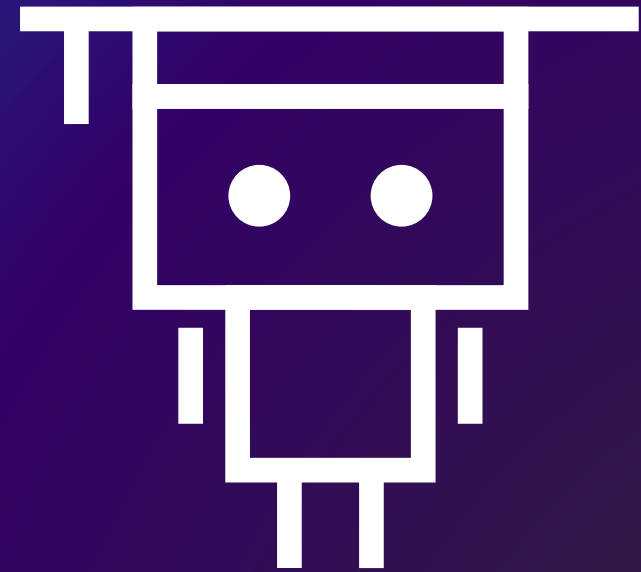
- Retrieval relies on semantic search
 - RAG effectiveness is heavily dependent on **quality/relevance of retrieved documents**
- **Increased latency** makes RAG less performant
 - Retrieval introduces latency, especially when querying from large databases
- **Computational cost** of inference is higher
 - With RAG, the number of input tokens in the prompt is larger (\$) and it requires additional compute for retrieval (\$), making it more expensive than simple inference



Next lesson

- This lesson covered retrieval augmented generation workflows.
- In the next lesson, you will learn about agentic applications.





Thank you!